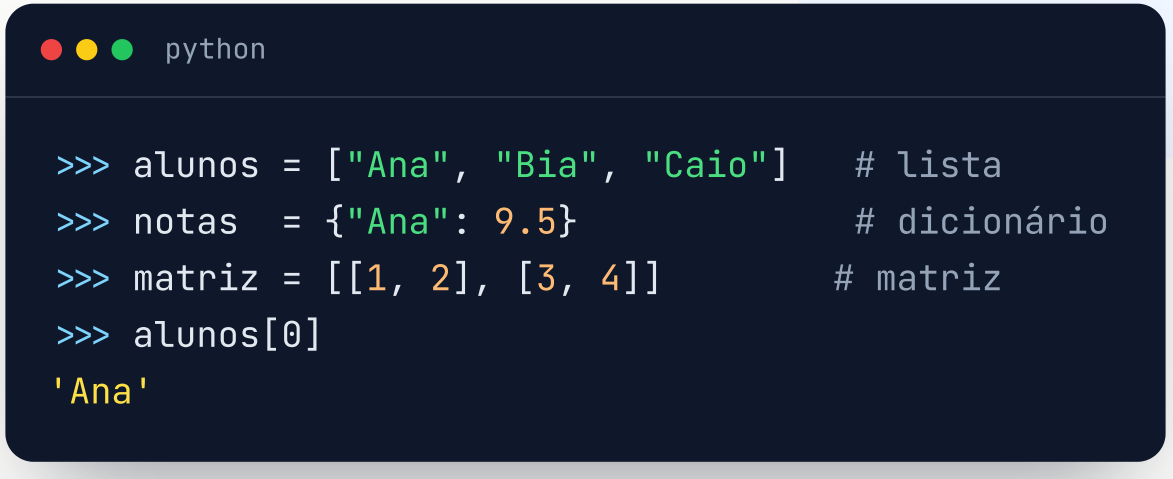
Introdução a Programação Estruturada em Python

Estruturas de Dados em Python

Matrizes • Listas • Tuplas • Conjuntos • Dicionários



Curso Superior de Ciência da Computação e Sistemas de Informação — 2º Semestre

Módulos 7–10 • Professores: **Luciano, Rafael e Turatti**

completa dos módulos 7 a 10 — conteúdo reordenado didaticamente para facilitar o aprendizado

Roteiro da aula

Conteúdo reordenado para facilitar o entendimento — cada parte se apoia na anterior

01

Módulo 8 — Estruturas de Dados

O panorama geral: sequência, lista, tupla e conjunto

a base de tudo

02

Módulo 9 — Listas a fundo

Os métodos de manipulação que você usará todos os dias

dominar a favorita

03

Módulo 7 — Matrizes

Listas de listas: dados em linhas e colunas

aplicação direta

04

Módulo 10 — Dicionários

Busca instantânea por chave: chave → valor

o mapa

Por que essa ordem? Uma matriz é uma lista de listas — por isso dominamos listas primeiro. E o dicionário fecha o ciclo resolvendo o que sequências não fazem bem: localizar um item sem conhecer sua posição.

PARTE 01 / 04

MÓDULO 8

Estruturas de Dados

Como o Python organiza informação: sequência, lista, tupla e conjunto

O que são estruturas de dados • Sequências e índices • Lista • Tupla • Conjunto (Set) • Comparativo

01

Conceito

O que é uma estrutura de dados?

É uma forma de organizar e guardar informações na memória para usá-las com eficiência.

- Analogia: a mochila guarda tudo junto; a estante organiza por posição — encontrar um livro fica muito mais rápido.
- Cada estrutura tem regras próprias: o que pode entrar, como acessar, o que pode mudar.
- Escolher bem a estrutura = código mais simples, mais rápido e mais fácil de manter.

python

```python
# a mesma informação,
# em estruturas diferentes

alunos = ["Ana", "Bia", "Caio"]
   # lista: ordenada, mutável
ponto  = (-23.55, -46.63)
   # tupla: fixa, imutável
vistos = {"Ana", "Caio"}
   # set: sem duplicatas
notas  = {"Ana": 9.5}
   # dict: busca por chave
```

Módulo 8 • Sequência

Sequências: itens numerados a partir do 0

Lista, tupla e string são sequências — todas compartilham indexação, fatiamento e tamanho.




```python
nome = "PYTHON"
nome[0]    # P → o 1º item tem índice 0
nome[-1]   # N → negativos contam do fim
nome[1:4]  # YTH → início incluso, fim excluso
len(nome)  # 6
'Y' in nome # True
```

indexação `s[i]` · fatiamento `s[i:j]` · `len(s)` · pertencimento `in` · `min` · `max`

Módulo 8 • Lista

Lista: a estrutura mais usada

- Coleção ordenada de itens, criada com colchetes `[ ]` ou com `list()`.
- Mutável: pode-se alterar, adicionar e remover itens depois de criada.
- Aceita tipos mistos e duplicatas; cada item tem um índice.

```python
compras = ["arroz", "feijão", "café"]
mistur  = [10, 3.14, "dez", True]  # tipos mistos

compras[1]            # feijão
compras[-1]           # café
compras[0] = "açúcar"  # mutável: substitui
compras               # [açúcar, feijão, café]
len(compras)          # 3
```

Ordenada | Mutável | Duplicatas ok

Índices `0..n-1`

Módulo 8 • Tupla

Tupla: a lista que não muda

```python
ponto = (3, 5)
rgb   = (255, 128, 0)
uma   = (42,)      # vírgula obrigatória!

x, y = ponto       # unpacking → x=3, y=5
ponto[0]           # 3 → leitura ok
ponto[0] = 10      # TypeError! item assignment
```

- Ordenada e com duplicatas, igual à lista — porém imutável.
- Criada com parênteses `( )` ou apenas vírgulas; tupla de 1 item exige vírgula final.
- Protege dados contra alterações acidentais e serve de chave de dicionário.

Quando usar?

Coordenadas (x, y), cores RGB, datas fixas, retorno múltiplo de funções:

```
def estatisticas(x):
    return min(x), max(x)
```

Módulo 8 • Conjunto

Conjunto (Set): itens únicos, sem ordem

- Não ordenado e sem duplicatas — perfeito para eliminar repetições.
- Mutável, mas sem índice: não existe `set[0]`.
- Tem operações matemáticas: união, interseção e diferença.

duplicatas desaparecem

```python
vogais = {"a", "e", "i", "e", "o"}
print(vogais)
{'a', 'e', 'i', 'o'}
vazio = set()   # {} cria dict, não set!
```

operações entre conjuntos

```python
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
a | b   # união        {1,2,3,4,5,6}
a & b   # interseção   {3,4}
a - b   # diferença    {1,2}
```

Lista × Tupla × Conjunto

Escolha rápida pela sintaxe e pelas regras de cada estrutura

CARACTERÍSTICA	Lista `[1, 2]`	Tupla `(1, 2)`	Conjunto `{1, 2}`
Ordenada por índice	✓	✓	✗
Mutável	✓	✗	✓
Aceita duplicatas	✓	✓	✗
Acesso por índice	✓	✓	✗
É uma sequência?	**Sim**	**Sim**	**Não**
Uso ideal	*coleção que muda*	*dados fixos*	*itens únicos*

PARTE 02 / 04

Módulo 9

Listas a fundo

Os métodos de manipulação que você vai usar todos os dias

Adicionar • **Remover** • **Buscar e contar** • **Ordenar** • **Funções prontas e cópias** • **List comprehension**

Módulo 9 • Listas

Adicionar itens: append, insert, extend

python

```python
times = ["Flamengo"]
times.append("Palmeiras")          # 1 item no fim
times.insert(0, "Corinthians")     # insere na posição 0
times.extend(["Santos", "Grêmio"]) # vários de uma vez

print(times)
['Corinthians', 'Flamengo', 'Palmeiras', 'Santos', 'Grêmio']
```

append × extend — a pegadinha clássica

python

```python
a = [1, 2]; a.append([3, 4])   # [1, 2, [3, 4]] → lista dentro da lista
b = [1, 2]; b.extend([3, 4])   # [1, 2, 3, 4]   → desmonta e adiciona
```

Módulo 9 • Listas

Remover itens: remove, pop, del, clear

```python
fila = ["ana", "bia", "caio", "dan"]

fila.remove("bia")    # pelo VALOR (1ª ocorrência)
fila.pop()            # último → devolve dan
fila.pop(0)           # posição 0 → devolve ana
del fila[0]           # pelo ÍNDICE, sem devolver
fila.clear()          # esvazia tudo → []
```

`remove()` exige que exista

Se o valor não está na lista, levanta **ValueError** — confira antes com `if valor in lista`.

`pop()` devolve o item

O removido pode ser guardado: **ótimo para filas e pilhas**.

`del` também fatia

`del lista[1:3]` remove um intervalo inteiro de uma vez.

Módulo 9 • Listas

Buscar e contar: index, count, in

```python
notas = [8, 7, 8, 10, 8, 5]

notas.index(10)   # 3 → posição da 1ª ocorrência
notas.count(8)    # 3 → quantas vezes aparece

10 in notas   # True → existe?
9 in notas    # False
```

Padrão de ouro

```python
if valor in lista:
    lista.remove(valor)
```

Verificar antes de agir evita **ValueError** e deixa o código à prova de erros.

O operador `in` funciona em **TODAS** as estruturas da aula: lista, tupla, set e as chaves do dicionário.

Módulo 9 • Listas

Ordenar: sort × sorted

`lista.sort()` — modifica a original

```python
nums = [3, 1, 4, 1, 5]
nums.sort()
print(nums)
[1, 1, 3, 4, 5]
nums.sort(reverse=True)
print(nums)
[5, 4, 3, 1, 1]
```

×

`sorted(lista)` — devolve NOVA lista

```python
nums = [3, 1, 4, 1, 5]
ordem = sorted(nums)
print(ordem)
[1, 1, 3, 4, 5]
print(nums)
[3, 1, 4, 1, 5]  # intacta!
```

Bônus: `palavras.sort(key=len)` ordena pelo número de letras — o parâmetro **key** aceita qualquer critério.

Módulo 9 • Listas

Funções prontas e cópias seguras

Σ Estatística instantânea (builtins)

```python
precos = [12.5, 8.0, 30.0, 5.75]
len(precos)   # 4      → quantidade
sum(precos)   # 56.25  → soma total
min(precos)   # 5.75
max(precos)   # 30.0
```

- `len()`
- `sum()`
- `min()`
- `max()`

Cópia: cuidado com o =

```python
copia = precos.copy()  # nova lista
alias  = precos        # NÃO é cópia!

alias.append(99.0)
print(precos)
[12.5, 8.0, 30.0, 5.75, 99.0] ⚠
print(copia)
[12.5, 8.0, 30.0, 5.75]  ✔
```

Módulo 9 • Bônus que vale ouro

List comprehension: uma linha, uma lista nova

A FÓRMULA

```
[expressão for item in sequência if condição]
```

elevar ao quadrado

```python
quadrados = [n ** 2
    for n in range(1, 6)]
print(quadrados)
[1, 4, 9, 16, 25]
```

filtrar pares

```python
pares = [n
    for n in range(10)
    if n % 2 == 0]
print(pares)
[0, 2, 4, 6, 8]
```

converter valores

```python
celsius = [0, 20, 37]
fahr = [c * 9 / 5 + 32
    for c in celsius]
print(fahr)
[32.0, 68.0, 98.6]
```

→ Equivale a um **for** que monta a lista item a item — e será a ferramenta para criar matrizes na próxima parte.

PARTE 03 / 04

Módulo 7

Matrizes

Listas de listas: dados organizados em linhas e colunas

Criar a matriz • Acessar `m[i][j]` • Percorrer com for • Somar e transpor

Módulo 7 • Matrizes

Uma matriz é uma lista de listas

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

linha → posição na **lista externa**
coluna → posição na **lista interna**

```python
matriz = [
    [1, 2, 3],   # linha 0
    [4, 5, 6],   # linha 1
    [7, 8, 9],   # linha 2
]

len(matriz)      # 3 → linhas
len(matriz[0])   # 3 → colunas
```

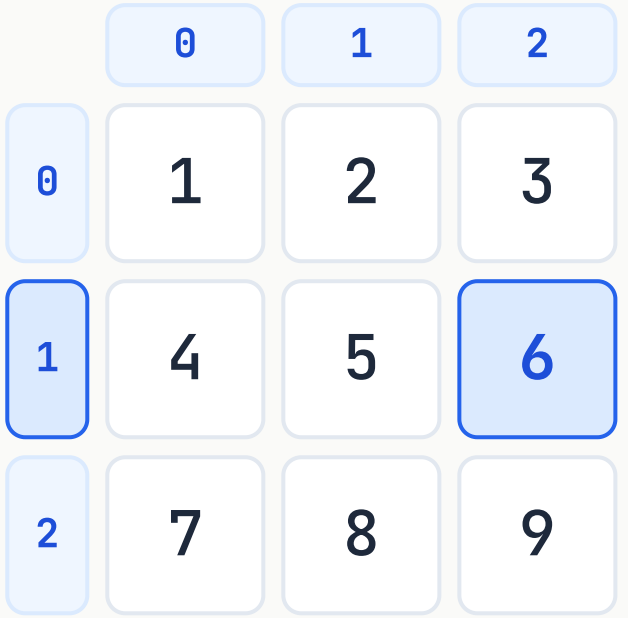
Cada linha é uma lista comum: tudo o que você já aprendeu sobre listas vale aqui dentro.

Módulo 7 • Matrizes

Acesso: `matriz[linha][coluna]`

```python
matriz = [[1, 2, 3],
          [4, 5, 6],
          [7, 8, 9]]

matriz[1]        # [4, 5, 6] → a linha 1 inteira
matriz[1][2]     # 6 → linha 1, coluna 2
matriz[0][-1]    # 3 → 1ª linha, última coluna
matriz[1][2] = 60  # mutável: troca o valor
```



linha 1 → depois → **coluna 2**

Leia de fora para dentro: primeiro `[linha]`, depois `[coluna]` — como coordenadas (`linha`, `coluna`) no mapa.

Módulo 7 • Matrizes

Percorrendo a matriz com for

por linha (o mais simples)

```python
for linha in matriz:
    print(linha)
[1, 2, 3]
[4, 5, 6]
[7, 8, 9]
```

elemento a elemento (for aninhado)

```python
for linha in matriz:
    for valor in linha:
        print(valor, end=" ")
    print()
```

com índices (enumerate)

```python
for i, linha in enumerate(matriz):
    for j, valor in enumerate(linha):
        print(f"m[{i}][{j}] = {valor}")
m[0][0] = 1  m[0][1] = 2 ...
```

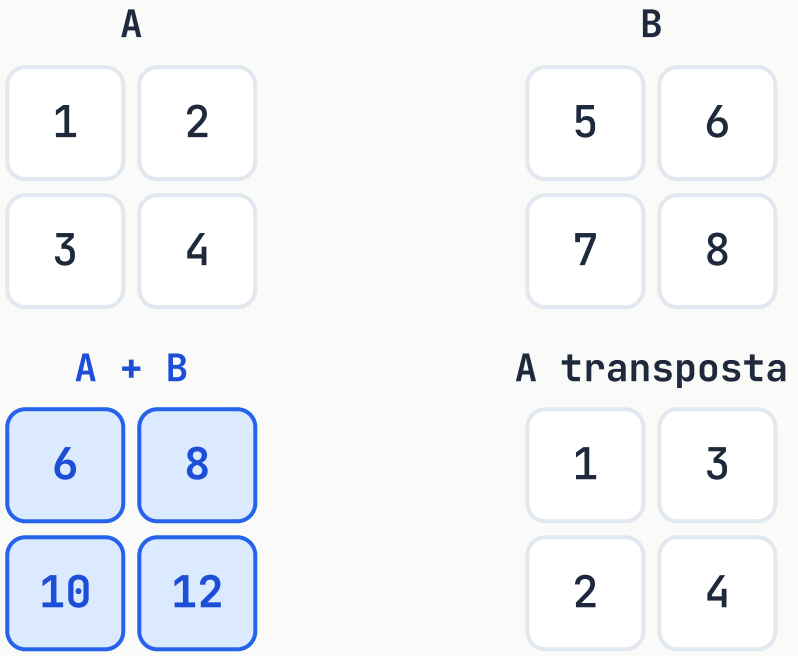
Módulo 7 • Matrizes

Operações clássicas: soma e transposta

```python
A = [[1, 2], [3, 4]]
B = [[5, 6], [7, 8]]

# soma: célula a célula
soma = [[A[i][j] + B[i][j]
         for j in range(2)]
        for i in range(2)]
[[6, 8], [10, 12]]

# transposta: linhas viram colunas
t = [[A[j][i] for j in range(2)]
     for i in range(2)]
[[1, 3], [2, 4]]
```



A soma exige matrizes de **mesmas dimensões**; a transposta troca linha por coluna (**`i` ↔ `j`**).

PARTE 04 / 04

MÓDULO 10

Dicionários

Do índice numérico à busca instantânea por chave

Conceito chave → valor • Criar e acessar • `get()` e `in` • Métodos principais • Percorrer e contar

04

Módulo 10 • Dicionários

Dicionário: cada valor tem um nome

Agenda telefônica: você procura pelo **NOME** (chave) e recebe o **TELEFONE** (valor) — sem folhear página por página.

```python
aluno = {
    "nome":  "Ana",
    "idade": 20,
    "curso": "Ciência da Computação",
}

aluno["nome"]    # Ana → acesso pela chave
aluno["idade"]   # 20
```

- Chave: imutável (str, número, tupla)
- Valor: qualquer tipo — até listas e dicts
- Cada chave aparece uma única vez

Módulo 10 • Dicionários

Criar, acessar e alterar com segurança

```python
estoque = {"arroz": 12, "feijão": 8}

estoque["arroz"]      # 12
estoque["macarrão"]   # KeyError!

estoque.get("macarrão")      # None (sem erro)
estoque.get("macarrão", 0)   # 0 (valor padrão)

estoque["arroz"] = 15   # altera existente
estoque["café"]  = 3    # cria novo par
"café" in estoque  # True
```

`estoque[chave]`

Erro **KeyError** se a chave não existe — **quebra o programa**.

VS

`estoque.get(chave, padrão)`

Devolve o **valor padrão** se não existe — **programa continua**.

Módulo 10 • Dicionários

Principais métodos do dicionário

```python
aluno = {"nome": "Ana", "idade": 20, "curso": "CC"}

aluno.keys()    # dict_keys([nome, idade, curso])
aluno.values()  # dict_values([Ana, 20, CC])
aluno.items()   # dict_items([(nome, Ana), ...])

aluno.pop("idade")           # 20 → remove e devolve
aluno.update({"ira": 8.7})   # insere/atualiza em lote
aluno.clear()                # → {}
```

`keys()`	`values()`	`items()`	`pop(chave)`	`update(outro)`	`clear()`
todas as chaves	todos os valores	pares (chave, valor)	remove e devolve	mescla/atualiza	esvazia o dict

Módulo 10 • Dicionários

Percorrer e construir dicionários

os dois jeitos de percorrer

```python
for chave in aluno:
    print(chave, aluno[chave])

for chave, valor in aluno.items():
    print(f"{chave}: {valor}")
```

exemplo clássico: contador de palavras

```python
texto = "abacaxi banana abacaxi"
contagem = {}

for p in texto.split():
    contagem[p] = contagem.get(p, 0) + 1

print(contagem)
{'abacaxi': 2, 'banana': 1}
```

`get(p, 0) + 1` é o padrão universal de contagem: se a chave não existe começa em **0**, depois soma **1**.

As 4 estruturas lado a lado

O mapa definitivo da aula

Característica	Lista	Tupla	Conjunto	Dicionário
Sintaxe	`[1, 2]`	`(1, 2)`	`{1, 2}`	`{"a": 1}`
Ordenada por índice	✓	✓	✗	✗
Mutável	✓	✗	✓	✓
Duplicatas	✓	✓	✗	só valores
Busca por	índice	índice	pertencimento	**chave**
Uso ideal	coleção que muda	dados fixos	itens únicos	mapear chave → valor

Como escolher?

- Ordem + alterações → **Lista**
- Registro imutável → **Tupla**
- Garantir itens únicos → **Set**
- Localizar por nome → **Dicionário**

Recapitulando e praticando

01

Estruturas: sequência, lista, tupla e set — cada uma com suas regras de ordenação e mutabilidade

02

Listas: append, pop, sort, in e comprehension resolvem quase tudo

03

Matrizes: matriz[linha][coluna] + for aninhado dominam linhas e colunas

04

Dicionários: chave → valor, com get() e items() no coração do dia a dia

Exercícios sugeridos

1. Crie uma matriz 3×3 e calcule a soma da diagonal principal.
2. Dada uma lista de notas, ordene com `sort()` e conte com `count()` quantas ficaram acima da média.
3. Conte a frequência de cada letra de uma palavra usando um dicionário e `get()`.

Bons estudos! Dúvidas → Professores Luciano, Rafael e Turatti